

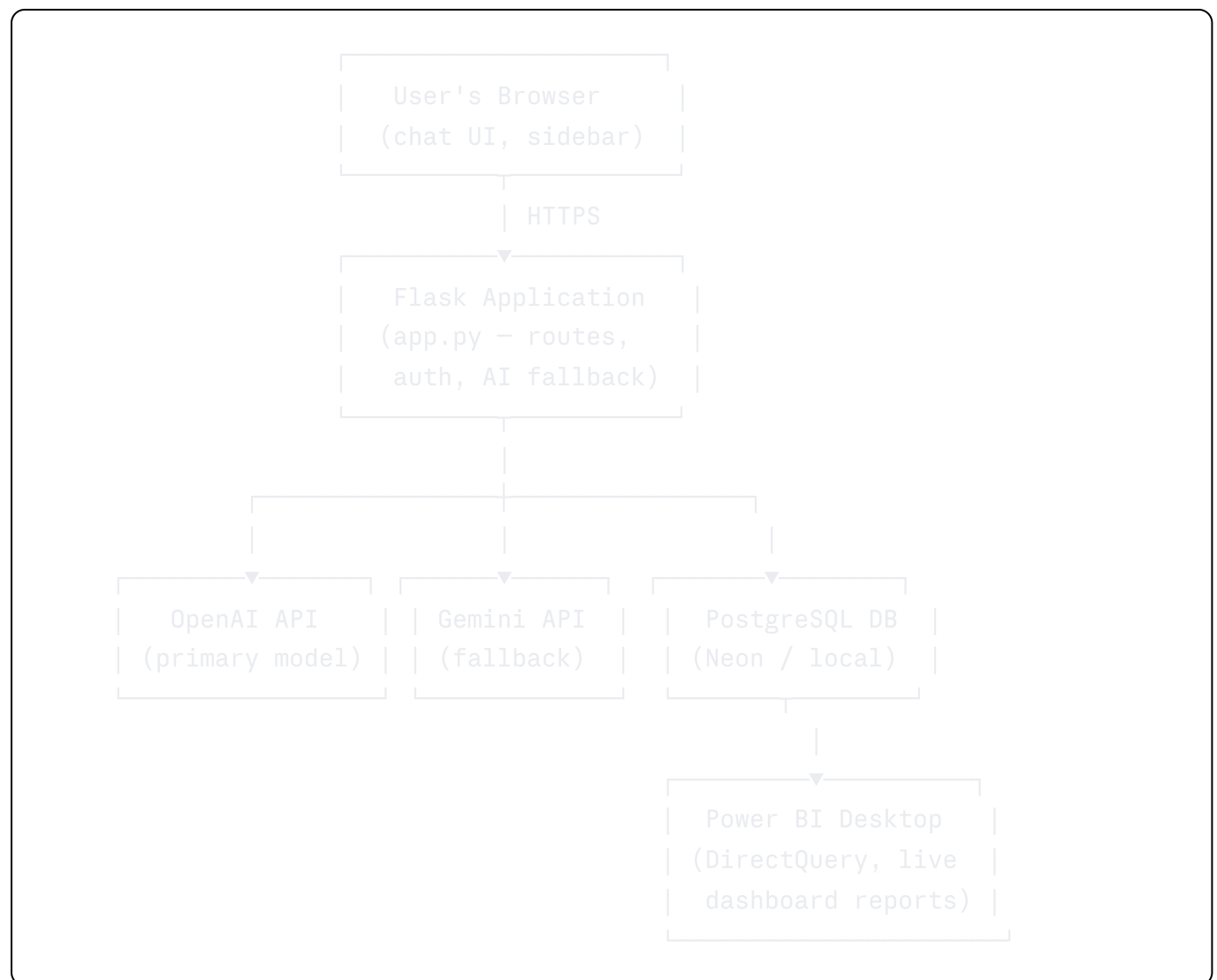
Technical Documentation — Flask AI Chat Application

1. Overview

This is a full-stack AI chat application built with Flask, featuring dual-AI failover (OpenAI + Google Gemini), encrypted message storage, real user authentication (email/password and Google OAuth), a ChatGPT-style interface with conversation history, structured error logging, and a live Power BI analytics dashboard connected via DirectQuery.

The project is deployed live on Render, backed by a PostgreSQL database hosted on Neon, with a separate local PostgreSQL instance used for development.

2. Architecture



Environment split:

- **Local development** (`python app.py`) → connects to a local PostgreSQL instance
- **Live deployment** (Render, `gunicorn app:app`) → connects to Neon PostgreSQL

The switch between these is automatic — the app checks for Render's own auto-injected `RENDER` environment variable at startup. No manual toggling or code edits are needed to

move between environments; both can run simultaneously without interfering with each other.

3. Tech Stack

Layer	Technology	Purpose
Backend framework	Flask	Routing, request handling
Production server	Gunicorn	Serves the app on Render
Database ORM	SQLAlchemy (via Flask-SQLAlchemy)	Database models and queries
Database (dev)	PostgreSQL (local, via pgAdmin)	Local development data
Database (prod)	PostgreSQL (Neon, serverless)	Live production data
AI — primary	OpenAI API (<code>gpt-5.6-luna</code>)	Chat completions
AI — fallback	Google Gemini API (<code>gemini-3.5-flash-lite</code>)	Chat completions if OpenAI fails
Authentication	Flask-Login, Werkzeug (password hashing)	Session management, email/password login
OAuth	Authlib	Google Sign-In
Encryption	<code>cryptography</code> (Fernet/AES)	Encrypts chat message content at rest
Frontend	HTML, CSS, vanilla JavaScript	Chat UI, sidebar, login page
Analytics	Power BI Desktop	Live dashboards via DirectQuery
Hosting	Render	Live web service
Version control	Git, GitHub (private repo)	Source control
CI/CD	GitHub Actions	Automated tests on push; scheduled ETL job
Testing	pytest	Automated smoke tests run in CI

4. Database Schema

The application uses four tables, all in one PostgreSQL database (`flask-chat-app`), replicated identically on both local Postgres and Neon.

user

Stores account records for both login methods.

Column	Type	Notes
id	Integer, PK	
email	String, unique	
password_hash	String, nullable	Empty for Google-only accounts
google_id	String, unique, nullable	Empty for email/password-only accounts
created_at	DateTime	

chat_message

Stores the actual conversation content. The **message** column is encrypted at rest — Power BI and any direct database query will only see ciphertext here, by design.

Column	Type	Notes
id	Integer, PK	
user_id	Integer, FK → user.id	
conversation_id	String	Groups messages into a single conversation thread
role	String	'user' or 'assistant'
message	Text	Encrypted (Fernet)
timestamp	DateTime	
source	String	Which AI answered: 'openai' or 'gemini'

request_log

Analytics-only metadata — deliberately contains **no message content**, so it's safe to connect Power BI to directly without touching decryption.

Column	Type	Notes
id	Integer, PK	
user_id	Integer, FK → user.id	
conversation_id	String	
timestamp	DateTime	
model_used	String, nullable	'openai', 'gemini', or null if both failed
success	Boolean	
response_time_ms	Integer	
category	String	Simple keyword-based classification (Programming / Data-SQL / AI / General / Other) — not real NLP

error_log

Captures unhandled application errors for the live "System Health / Bug Report" dashboard page. Routine HTTP errors (404, 405, etc.) are deliberately excluded — only genuine application exceptions are logged here.

Column	Type	Notes
id	Integer, PK	
timestamp	DateTime	
endpoint	String	Which route the error occurred on
error_type	String	Python exception class name
error_message	Text	Truncated traceback (max 500 chars)
user_id	Integer, FK → user.id, nullable	

daily_stats

Produced by the ETL pipeline (Section 8), not written to directly by the application. One row per calendar day, upserted daily.

Column	Type	Notes
id	Integer, PK	
stat_date	Date, unique	
total_requests	Integer	
successful_requests	Integer	
failed_requests	Integer	
error_rate	Numeric(5,2)	
avg_response_time_ms	Integer, nullable	
top_category	String, nullable	Most common category that day
total_errors	Integer	
run_at	Timestamp	When the ETL job last updated this row

5. Security Design

- **Passwords** are never stored in plain text — hashed with Werkzeug's `generate_password_hash` (PBKDF2-based).
- **Chat message content** is encrypted at rest using Fernet symmetric encryption before being written to the database. The encryption key is stored only in environment variables, never in code or version control.
- **Google OAuth** uses the standard authorization code flow via Authlib; no Google password ever touches this application.
- **Secrets management:** all API keys, encryption keys, and database credentials are stored in environment variables (`.env` locally, Render's environment variable settings in production) and excluded from version control via `.gitignore`. A `.env.example` file documents required variables without exposing real values.
- **Session security:** Flask-Login manages authenticated sessions via a dedicated `FLASK_SECRET_KEY`, separate from other application secrets.

6. AI Fallback Logic

The `/chat` endpoint calls OpenAI first. If that request fails for any reason (rate limit, quota exhaustion, API error, network issue), the exception is caught, logged, and the same request is automatically retried against Google Gemini — transparently to the user. If both providers fail, the failure is logged to `request_log` (with `success=False`) and `error_log`, and the user receives a clear error message rather than a raw exception.

This provides resilience against any single provider's outages or rate limits without requiring user intervention.

7. Deployment

Local development

```
bash

python -m venv venv
source venv/Scripts/activate # Windows Git Bash
pip install -r requirements.txt
python app.py
```

Requires a local PostgreSQL instance and a `.env` file (see `.env.example` for required variables).

Production (Render)

- **Build command:** `pip install -r requirements.txt`
- **Start command:** `gunicorn app:app`
- **Database:** Neon PostgreSQL (connection string set via the `DATABASE_URL` equivalent environment variable)
- **Environment variables:** set directly in Render's dashboard, mirroring the `.env.example` structure

Known limitation: Render's free tier spins down the service after ~15 minutes of inactivity. The first request after idle time may take 30–60 seconds while the service restarts. This is an accepted tradeoff of free-tier hosting, not an application defect.

8. CI/CD and ETL Pipeline

Continuous Integration (CI)

Every push to `main` (and every pull request) automatically triggers a GitHub Actions workflow (`.github/workflows/ci.yml`) that:

1. Spins up a fresh, temporary PostgreSQL instance (isolated from both local and production data)
2. Installs the project's dependencies from `requirements.txt`
3. Runs a small automated test suite (`tests/test_app.py`) covering: the login page loads, unauthenticated requests to `/chat` are correctly rejected, and password validation rejects weak passwords

This catches broken code — syntax errors, missing imports, regressions in core routes — before it's ever assumed to work, independent of manual testing.

Continuous Deployment (CD)

Render is connected directly to the GitHub repository and automatically rebuilds and redeploys the live application on every push to `main`, using the same `requirements.txt` and `gunicorn app:app` start command described in Section 7.

ETL Pipeline

A standalone script, `etl_daily_stats.py`, extracts raw rows from `request_log` and `error_log`, aggregates them into daily summaries, and loads the result into a new `daily_stats` table:

- **Extract** — raw request and error rows for a given day
- **Transform** — aggregated into per-day totals: request volume, success/failure counts, error rate, average response time, and the most common question category
- **Load** — upserted into `daily_stats` (one row per day; re-running the script safely updates the same day's row rather than duplicating it)

This script is intentionally independent of `app.py` — it only needs a database connection, not the full Flask application context — and runs against either database using the same environment-variable pattern as the rest of the project (`DATABASE_URL` for Neon, falling back to local Postgres otherwise).

A second GitHub Actions workflow (`.github/workflows/etl.yml`) runs this script automatically once a day via a scheduled cron trigger, targeting the production Neon database through a repository secret (`NEON_DATABASE_URL`) — never committed to the codebase. It can also be triggered manually from GitHub's Actions tab for on-demand runs.

9. Power BI Analytics

Two separate `.pbix` files are maintained:

- `flaskchatapp_analytics_dashboard.pbix` — connects to local PostgreSQL, used during development
- `flaskchatapp_dashboard_LIVE_neon.pbix` — connects to the live Neon database, reflects real production usage

Both use **DirectQuery** mode (not Import) against the `request_log` and `error_log` tables only — never `chat_message`, which is encrypted and would be unreadable to Power BI regardless. Automatic Page Refresh is configured at 10-second intervals.

Dashboard pages:

1. **Executive Overview** — total conversations, messages, active users, average response time, error rate, usage trend
2. **Model Analytics** — request volume and response time by AI provider, usage split
3. **Categories** — message volume by question category

4. **System Health & Bug Report** — success/error rates, average response time, error breakdown by type, and a live table of recent error details

Known limitation: Power BI's automatic refresh only runs while Power BI Desktop is actively open on the operator's machine. It is not a publicly hosted, always-on dashboard. Achieving that would require Power BI Service with a Pro license and a data gateway — a natural next step given the current DirectQuery-based architecture, but outside the current scope.

10. Future Improvements

- Add OpenAI billing to exercise the primary AI path in production (currently often falling back to Gemini)
- Publish the Power BI report to Power BI Service for genuinely public, always-on access
- Upgrade Render to a paid tier to eliminate cold-start delays
- Replace the keyword-based question categorization with a proper NLP/ML classifier
- Add a Power BI page reading from the new `daily_stats` table produced by the ETL pipeline
- Expand the CI test suite beyond the current smoke tests (e.g. covering the AI fallback logic, encryption round-tripping, and conversation history endpoints)